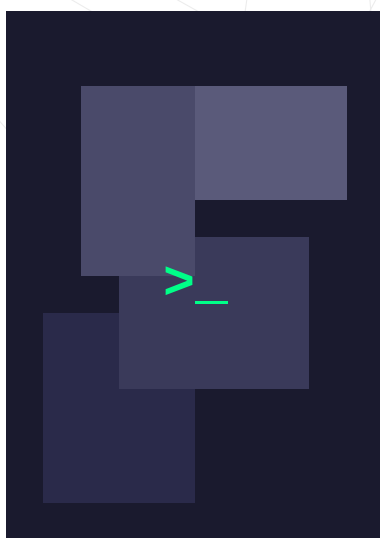




Claude Code — Módulo 00

CodeFlow

Resumen:

Este documento contiene los ejercicios del Módulo 00 de Claude Code.
Domina tu agente de desarrollo: desde /init hasta SDK.

Versión: 2.0

Contenido

- I** Introducción
- II** Reglas generales
- III** Reglas específicas del módulo
- IV** Ejercicio 00: Init & Context
- V** Ejercicio 01: Session Management
- VI** Ejercicio 02: HookGuard
- VII** Ejercicio 03: PlayBot
- VIII** Ejercicio 04: SDK Bridge
- IX** Evaluación

Capítulo I

Introducción

Claude Code es una herramienta de línea de comandos creada por Anthropic que convierte a Claude en un agente de desarrollo autónomo con acceso directo a tu sistema de archivos, terminal y flujo de trabajo.

El objetivo de este módulo es que aprendas a controlar, automatizar y extender Claude Code como herramienta profesional de desarrollo. No basta con usarlo: debes entender cómo funciona su bucle interno, cómo interceptarlo con hooks, cómo gestionar el contexto, y cómo integrarlo programáticamente en tus propios proyectos.

Cada ejercicio cubre una capa distinta de control, desde la configuración básica hasta la integración con SDK desde C++. Al final del módulo serás capaz de diseñar flujos de trabajo donde Claude Code trabaje para ti de forma supervisada y eficiente.



Lee el módulo completo antes de empezar. De verdad, hazlo. Entender el panorama general te ahorrará tiempo en cada ejercicio.

Capítulo II

Reglas generales

Requisitos del entorno

- Claude Code instalado y autenticado (cuenta Pro mínimo).
- Node.js >= 18 instalado (necesario para SDK y MCP).
- jq instalado en el sistema (sudo apt install jq o brew install jq).
- Git inicializado en cada directorio de ejercicio.
- Editor de texto a tu elección (vim, nvim, VSCode, etc.).

Estructura de entrega

Los directorios se nombrarán: ex00, ex01, ex02, ex03, ex04. Cada ejercicio incluye un starter pack descargable. No modifiques la estructura de carpetas del starter — añade tus archivos dentro de ella.

Compilación (ejercicios con código C++)

- Compila con c++ y las flags -Wall -Wextra -Werror.
- Tu código debe compilar con -std=c++17 o superior.
- Cada programa debe incluir un Makefile con: \$(NAME), all, clean, fclean, re.

Estructura de cada ejercicio

Cada ejercicio tiene una parte obligatoria (Mandatory) y una parte opcional (Bonus). La parte obligatoria debe funcionar completamente para poder optar a la puntuación del bonus. El bonus nunca compensa un mandatory incompleto.



Cada ejercicio debe realizarse en una rama de git separada. Haz commit de checkpoint ANTES de dejar que Claude Code modifique tu código. Si no puedes explicar qué hace cada línea, no lo entregues.

Capítulo III

Reglas específicas del módulo

Claude Code es una herramienta poderosa. Con gran poder viene gran responsabilidad (y gran factura de API si no controlas el contexto).

- Debes demostrar que entiendes cada concepto, no solo que lo has ejecutado.
- En cada ejercicio debes incluir un archivo `NOTES.md` con una explicación en tus propias palabras de lo que hace y por qué funciona.
- No se permite copiar configuraciones de internet sin entenderlas. Durante la evaluación se te preguntará por qué has tomado cada decisión.
- Claude Code debe usarse en modo interactivo (no autónomo) salvo que el ejercicio lo indique explícitamente.
- Cada ejercicio incluye un starter pack. Descárgalo antes de empezar.



El objetivo no es que Claude Code haga tu trabajo. El objetivo es que TÚ controles a Claude Code. Si durante la evaluación no puedes explicar tu configuración, tu nota será 0.



Es recomendable leer cada ejercicio completo antes de empezar. Los ejercicios son progresivos: cada uno construye sobre el anterior.

Capítulo IV

Ejercicio 00: Init & Context

> _	Ejercicio: 00	Init & Context
	Directorio: ex00/	
	Archivos: CLAUDE.md, NOTES.md, screenshots/	
	Prohibido: Ninguno	

Antes de pedirle nada a Claude Code, necesitas que sepa dónde está. Este ejercicio cubre la inicialización de un proyecto y el control del contexto.

Starter pack incluido:

```
ex00/  
■■■■ Makefile  
■■■■ inc/  
■ ■■■■ Calculator.hpp  
■■■■ src/  
■ ■■■■ main.cpp  
■ ■■■■ Calculator.cpp  
■■■■ tests/  
■ ■■■■ basic_test.cpp  
■■■■ NOTES.md
```

Parte obligatoria

1. Inicialización

- Ejecuta /init dentro del directorio del starter pack.
- Examina el CLAUDE.md generado. Documenta en NOTES.md: qué ha detectado Claude, qué le falta, qué cambiarías.

2. Personalización del CLAUDE.md

Modifica el CLAUDE.md para incluir al menos:

- Reglas de compilación: flags, estándar C++, estructura de directorios.
- Restricciones de estilo: nombres de variables, formato de comentarios.
- Limitación de directorios: Claude solo puede modificar src/ y inc/.
- Instrucción de idioma: respuestas en español.

3. Referencias con @

- Usa `@src/Calculator.cpp` para que Claude solo tenga en cuenta un archivo.
- Usa `@tests/` para limitar el contexto a la carpeta de tests.
- Captura pantallazos que demuestren que Claude respeta esas limitaciones.

Output esperado

- CLAUDE.md personalizado con las 4 secciones indicadas.
- NOTES.md con análisis crítico del CLAUDE.md original vs personalizado.
- Carpeta screenshots/ con al menos 3 capturas demostrando el uso de `@`.

Bonus

- Añade una sección de "errores frecuentes" al CLAUDE.md con patrones que Claude debe evitar (ej: no usar `printf`, no modificar el `Makefile`).
- Crea un segundo CLAUDE.md en tests/ con reglas específicas para testing.



El CLAUDE.md es el contrato entre tú y Claude Code. Si no está bien definido, Claude hará lo que le parezca. Un buen CLAUDE.md es la diferencia entre un agente útil y uno peligroso.

Capítulo V

Ejercicio 01: Session Management

>_	Ejercicio: 01	Session Management
	Directorio: ex01/	
	Archivos: NOTES.md, session_log.txt, screenshots/	
	Prohibido: Ninguno	

Claude Code tiene una ventana de contexto limitada. Si no la gestionas, empezará a olvidar cosas, a repetirse, o a consumir tokens innecesariamente. Este ejercicio te enseña a controlar esa ventana.

Starter pack incluido:

ex01/

■■■ session_scenario.md
■■■ session_log.template.txt
■■■ NOTES.md

Parte obligatoria

1. Escenario guiado (obligatorio para todos)

Ejecuta el siguiente escenario completo usando el proyecto del ex00:

- Paso 1: Pide a Claude que analice la estructura del proyecto.
- Paso 2: Pide que localice la función principal de Calculator.
- Paso 3: Pide una mejora pequeña (añadir un método nuevo).
- Paso 4: Pide que ejecute make.
- Paso 5: Introduce un error deliberado en el código.
- Paso 6: Pide que lo diagnostique y corrija.
- Paso 7: Ejecuta /compact.
- Paso 8: Pide una nueva modificación relacionada con el paso 3.
- Paso 9: Observa y documenta qué recuerda y qué necesita releer.

2. Compact con instrucciones

- Ejecuta /compact [instrucciones] dándole prioridades específicas de qué información conservar.
- Compara el resultado con el compact normal del paso 7.

3. Clean & Escape

- Demuestra el uso de `/clean` para limpiar el historial.
- Demuestra el uso de `Esc Esc` para interrumpir una operación en curso.
- Documenta en `NOTES.md`: ¿cuándo usarías `/compact` vs `/clean`? ¿En qué situación es crítico el doble escape?

Output esperado

- `session_log.txt` con el registro completo del escenario (9 pasos).
- `NOTES.md` con observaciones antes/después de `/compact`: qué recordó, qué perdió, qué tuvo que releer.
- Al menos 2 estrategias propias para minimizar el consumo de tokens.

Bonus

- Monitoriza el consumo de tokens durante la sesión y genera una tabla comparativa: operación → tokens consumidos.
- Identifica qué tipos de operaciones consumen más contexto (lectura de ficheros grandes, outputs de bash, etc.).



El contexto es tu recurso más valioso en Claude Code. Gestionarlo bien es como gestionar la memoria en C: si no lo controlas, te explota en la cara.

Capítulo VI

Ejercicio 02: HookGuard

>_	Ejercicio: 02	HookGuard
	Directorio: ex02/	
	Archivos: settings.json, hooks/, logs/, NOTES.md	
	Prohibido: Ninguno	

Los hooks son el mecanismo que convierte a Claude Code de herramienta interactiva a agente supervisado. En este ejercicio implementarás hooks que protejan y monitoricen el trabajo de Claude Code.

Starter pack incluido:

```
ex02/
■■■■ settings.json
■■■■ hooks/
■   ■■■■ guard.sh
■   ■■■■ compiler_check.sh
■■■■ logs/
■   ■■■■ .gitkeep
■■■■ sample_hooks/
■   ■■■■ bash_post.json
■   ■■■■ write_post.json
■   ■■■■ read_post.json
■■■■ NOTES.md
```

Parte obligatoria

1. Inspección con jq (SIEMPRE primero)

Antes de escribir cualquier hook funcional:

- Crea un hook PostToolUse con matcher "*" que vuelque stdin a un archivo log (jq . > hook-log.json).
- Haz que Claude use al menos 3 herramientas distintas (Read, Write, Bash).
- Examina los JSON resultantes. Compáralos con los sample_hooks/ del starter. Documenta en NOTES.md la estructura de cada uno.

2. PreToolUse — El guardián

Implementa hooks/guard.sh que:

- Bloquee cualquier comando bash que contenga `rm -rf` o `sudo`.
- Loguee todos los comandos bash intentados en `logs/commands.log` con timestamp.

```
# Estructura esperada en settings.json
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "bash hooks/guard.sh"
      }]
    }]
  }
}
```



En una configuración real se recomienda usar rutas absolutas para hooks. Este starter usa rutas relativas por simplicidad didáctica. Como bonus avanzado, genera `settings.local.json` con rutas absolutas.

3. PostToolUse — El verificador

Implementa `hooks/compiler_check.sh` que:

- Después de cada Write en `.cpp` o `.hpp`, ejecute `make`.
- Si falla, el error se pase de vuelta a Claude para que corrija.
- Loguee cada resultado (OK/FAIL + timestamp) en `logs/compile.log`.

Output esperado

- `settings.json` con ambos hooks configurados (Pre y Post).
- `hooks/guard.sh` funcional — bloquea `rm -rf` y `sudo`.
- `hooks/compiler_check.sh` funcional — compila tras cada edición.
- `logs/commands.log` con al menos 5 entradas con timestamp.
- `logs/compile.log` con al menos 3 entradas (OK y FAIL).
- `NOTES.md` explicando la estructura del JSON de cada herramienta.

Bonus

- Añade un hook Notification que envíe un mensaje a un archivo `notifications.log` cuando Claude lleve más de 60 segundos idle.
- Implementa un hook Stop que ejecute `make clean && make` al final de cada turno de Claude y loguee el resultado.
- Genera un `settings.local.json` que use rutas absolutas en todos los hooks. Documenta en `NOTES.md` por qué las rutas absolutas son más seguras que las relativas en un entorno de producción.



Siempre inspecciona con jq primero (Parte 1) antes de escribir el hook funcional. No asumas la estructura del JSON — verifícala. Es el equivalente a hacer printf de debug antes de codificar la lógica.

Capítulo VII

Ejercicio 03: PlayBot

> _	Ejercicio: 03	PlayBot
	Directorio: ex03/	
	Archivos: mcp_config.json, NOTES.md, reports/, screenshots/	
	Prohibido: Ninguno	

MCP (Model Context Protocol) permite a Claude Code conectarse con herramientas externas. Playwright es un framework de automatización de navegadores. Combinados, Claude Code puede navegar la web, testear interfaces y extraer datos de páginas reales.

Starter pack incluido:
ex03/
■■■ mcp_config.json
■■■ reports/
■ ■■■ .gitkeep
■■■ screenshots/
■ ■■■ .gitkeep
■■■ NOTES.md

Parte obligatoria

1. Configuración MCP

- Instala el servidor MCP de Playwright.
- Configura Claude Code para que lo detecte (mcp_config.json).
- Verifica que Claude Code lista las herramientas del MCP disponibles.
- Captura pantallazo de la verificación.

2. Extracción de datos

Pide a Claude Code que usando Playwright:

- Navegue a una web pública (ej: tu perfil de GitHub).
- Extraiga información específica (nombre de repos, bio, lenguajes).
- Genere un informe en reports/github_profile.md con los datos.

3. Test automatizado

Diseña un test que Claude ejecute via Playwright:

- Verificar que una página carga correctamente (status 200).
- Comprobar que un elemento específico existe en la página.
- Reportar resultado (PASS/FAIL + timestamp) en reports/test_results.md.

Output esperado

- mcp_config.json funcional con Playwright configurado.
- reports/github_profile.md con datos reales extraídos.
- reports/test_results.md con al menos 2 tests (PASS/FAIL).
- screenshots/ con capturas del proceso.
- NOTES.md explicando el patrón MCP: configurar, verificar, usar.

Bonus

- Configura un segundo servidor MCP (filesystem, GitHub, etc.) y demuestra que Claude puede usar ambos en la misma sesión.
- Combina un hook PostToolUse del ex02 con el MCP de Playwright: tras cada edición web, loguea qué página se visitó.



No uses Playwright para acceder a contenido privado, cuentas personales con credenciales, ni para hacer scraping masivo. Solo webs públicas.



MCP es el sistema de plugins de Claude Code. Playwright es solo un ejemplo. El patrón que aprendes aquí (configurar, verificar, usar) es el mismo para cualquier servidor MCP.

Capítulo VIII

Ejercicio 04: SDK Bridge

>_	Ejercicio: 04	SDK Bridge
	Directorio: ex04/	
	Archivos: Makefile, main.cpp, CodeReviewer.{cpp,hpp}, bridge.mjs, NOTES.md	
	Prohibido: Ninguno	

El Agent SDK permite llamar a Claude Code programáticamente desde tu propio código. En este ejercicio construirás un puente entre C++ y Claude Code: tu programa en C++ lanzará Claude Code via SDK para que revise código automáticamente.

Starter pack incluido:

```
ex04/
■■■■ Makefile
■■■■ main.cpp
■■■■ CodeReviewer.hpp
■■■■ CodeReviewer.cpp
■■■■ bridge.mjs
■■■■ sample_report.json
■■■■ test_files/
■   ■■■■ leak.cpp
■   ■■■■ unittest.cpp
■   ■■■■ logic_bug.cpp
■■■■ logs/
■   ■■■■ .gitkeep
■■■■ NOTES.md
```

Parte obligatoria

Este ejercicio se divide en pasos incrementales. Completa cada uno antes de avanzar al siguiente.

Paso 1: Leer JSON desde C++

- Lee sample_report.json desde tu programa C++.
- Parsea el JSON y muestra los issues por stdout de forma legible.
- No necesitas el SDK todavía — usa el archivo de ejemplo incluido.

```
// Formato de sample_report.json
{
  "file": "leak.cpp",
  "issues": [
    {
      "line": 4,
      "type": "memory_leak",
      "message": "new sin delete correspondiente"
    }
  ],
  "summary": "1 issue encontrado"
}
```

Paso 2: Script SDK

- Instala: `npm install @anthropic-ai/claude-agent-sdk`.
- Completa `bridge.mjs` para que reciba la ruta de un `.cpp` como argumento, lo envíe a Claude Code para análisis, y genere un JSON con el mismo formato que `sample_report.json`.
- Usa `allowedTools: ["Read"]` para limitar qué puede hacer Claude.

Paso 3: Conectar C++ con SDK

- Modifica `CodeReviewer` para que lance `bridge.mjs` con `system()`.
- Lee el JSON generado y muéstralo por `stdout`.
- Gestiona errores: fichero no encontrado, SDK no disponible, JSON malformado.

Paso 4: Validación con `test_files/`

- Ejecuta tu programa sobre los 3 archivos de `test_files/`.
- `leak.cpp` tiene un `new` sin `delete`.
- `uninit.cpp` tiene una variable sin inicializar.
- `logic_bug.cpp` tiene un error de lógica.
- Verifica que Claude detecta los issues en cada uno.

Output esperado

- `CodeReviewer` compilado y funcional.
- `bridge.mjs` que genera JSON válido.
- Ejecución exitosa sobre los 3 `test_files`.
- `NOTES.md` explicando el flujo completo: C++ → `system()` → `bridge.mjs` → SDK → Claude → JSON → C++ muestra resultado.

Bonus

- Integra un hook `Stop` del `ex02`: cuando Claude termine, ejecuta automáticamente `CodeReviewer` sobre los ficheros modificados y loguea el resultado en `logs/review.log` con timestamp.
- Sustituye `system()` por `fork()/exec()` para lanzar `bridge.mjs`. `system()` es más simple pero menos seguro; `fork()/exec()` es más robusto y demuestra conocimiento de programación de sistemas.

Bonus final: notificación remota

Lleva el flujo un paso más allá: en lugar de loguear el resultado del review en un archivo .log, envíalo como mensaje a Telegram o Slack.

- Crea un bot de Telegram con @BotFather y obtén el token.
- Modifica el hook Stop para que, tras ejecutar CodeReviewer, envíe el resumen via curl al bot de Telegram.
- El mensaje debe incluir: nombre del fichero, número de issues, timestamp y resumen de los errores.
- Alternativa: usa un webhook de Slack si lo prefieres.

```
# Ejemplo: enviar resultado a Telegram desde el hook
TOKEN="tu_bot_token"
CHAT_ID="tu_chat_id"
MSG="Review: $FILE - $ISSUES issues encontrados"
curl -s -X POST "https://api.telegram.org/bot$TOKEN/sendMessage" \
  -d chat_id="$CHAT_ID" -d text="$MSG"
```



Este bonus cierra el círculo completo del módulo: Claude edita código, el hook lanza el review via SDK, el resultado te llega al móvil. Lanzas el proceso, te vas al CrossFit, y te llega el resultado.



Este ejercicio combina TODO lo aprendido en el módulo: CLAUDE.md (ex00), gestión de contexto (ex01), hooks (ex02), y ahora SDK (ex04). Si has entendido cada pieza, el puzzle se monta solo.



El SDK es TypeScript/Python. Desde C++ se invoca via `system()` o `fork()/exec()`. Es un workaround válido y usado en producción.

Capítulo IX

Evaluación

Entrega tu trabajo en tu repositorio Git. Solo se evaluará el trabajo dentro de tu repositorio durante la defensa.

Durante la evaluación se te puede pedir que:

- Expliques cada línea de tus hooks y configuraciones.
- Modifiques un hook en vivo para añadir una funcionalidad nueva.
- Demuestres que entiendes el flujo completo: desde /init hasta SDK.
- Expliques las decisiones de tu CLAUDE.md y por qué elegiste esas reglas.
- Ejecutes tu CodeReviewer sobre un fichero nuevo que el evaluador te proporcione.

El NOTES.md de cada ejercicio es OBLIGATORIO. Sin él, el ejercicio se califica como 0 independientemente de que funcione.



El objetivo de este módulo no es que Claude Code trabaje por ti. Es que tú demuestres que controlas a Claude Code. Si no puedes explicar tu trabajo, no es tu trabajo.



¡Por Odín, por Thor! ¡Usa tu cerebro! Claude Code es una herramienta, no un sustituto de tu capacidad de razonamiento.